Use standard J2SE APIs in the **java.text** package to correctly format or parse dates, numbers, and currency values for a specific locale; and, given a scenario, determine the appropriate methods to use if you want to use the default locale or a specific locale. Describe the purpose and use of the **java.util.Locale** class.

Formatting and parsing a **Date** using default formats

Every locale has four default formats for formatting and parsing dates. They are called `SHORT`, `MEDIUM`, `LONG`, and `FULL`. The `SHORT` format consists entirely of numbers while the `FULL` format contains most of the date components. There is also a default format called `DEFAULT` and is the same as `MEDIUM`.

This example formats dates using the default locale (which is "`ru_RU`"). If the example is run in a different locale, the output will not be the same:

```
import static java.lang.System.out;
import java.text.DateFormat;
import java.text.ParseException;
import java.util.Date;
import java.util.Locale;

public class DateFormatExample1 {

    public static void main(String[] args) {
        // Format
        Date date = new Date();

        String s = DateFormat.getDateInstance(DateFormat.SHORT).format(date);
        out.println("DateFormat.SHORT : " + s);

        s = DateFormat.getDateInstance(DateFormat.MEDIUM).format(date);
        out.println("DateFormat.MEDIUM : " + s);

        s = DateFormat.getDateInstance(DateFormat.LONG).format(date);
        out.println("DateFormat.LONG : " + s);

        s = DateFormat.getDateInstance(DateFormat.FULL).format(date);
        out.println("DateFormat.FULL : " + s);

        s = DateFormat.getDateInstance().format(date);
        out.println("default : " + s);

        s = DateFormat.getDateInstance(DateFormat.DEFAULT).format(date);
        out.println("DateFormat.DEFAULT : " + s);

        // Parse
        try {
            date = DateFormat.getDateInstance(DateFormat.DEFAULT).parse("08.02.2005");
            out.println("Parsed date : " + date);
        } catch (ParseException e) {
            out.println(e);
        }
    }
}
```

The output:

```
DateFormat.SHORT : 08.02.05
DateFormat.MEDIUM : 08.02.2005
DateFormat.LONG : 8 Февраль 2005 г.
DateFormat.FULL : 8 Февраль 2005 г.
default : 08.02.2005
DateFormat.DEFAULT : 08.02.2005
Parsed date : Tue Feb 08 00:00:00 EET 2005
```

`DateFormat` is an `abstract` class for date/time formatting subclasses which formats and parses dates or time in a language-independent manner. The date/time formatting subclass, such as `SimpleDateFormat`, allows for formatting (i.e., date -> text), parsing (text -> date), and normalization. The date is represented as a `Date` object or as the milliseconds since January 1, 1970, 00:00:00 GMT.

`DateFormat` helps you to format and parse dates for any locale. Your code can be completely independent of the locale conventions for months, days of the week, or even the calendar format: lunar vs. solar.

To format a date for the current `Locale`, use one of the `static` factory methods:

```
myString = DateFormat.getDateInstance().format(myDate);
```

To format a date for a different `Locale`, specify it in the call to `getDateInstance()`:

```
DateFormat df = DateFormat.getDateInstance(DateFormat.LONG, Locale.US);
```

You can use a `DateFormat` to parse also:

```
myDate = df.parse(myString);
```

NOTE, the `parse(...)` method throws `ParseException` if the beginning of the specified string cannot be parsed. The `ParseException` is a checked exception, so parsing always should be done inside `try-catch` block.

Formatting and parsing a date for a **`Locale`**

To format and parse in a particular locale, specify the locale when creating the `DateFormat` object:

```
import static java.lang.System.out;
import java.text.DateFormat;
import java.text.ParseException;
import java.util.Date;
import java.util.Locale;

public class DateFormatExample2 {

    public static void main(String[] args) {
        //  Format
        Date date = new Date();
        Locale locale = Locale.US;

        String s = DateFormat.getDateInstance(DateFormat.SHORT, locale).format(date);
        out.println("DateFormat.SHORT : " + s);

        s = DateFormat.getDateInstance(DateFormat.MEDIUM, locale).format(date);
        out.println("DateFormat.MEDIUM : " + s);
```

```
        s = DateFormat.getDateInstance(DateFormat.LONG, locale).format(date);
        out.println("DateFormat.LONG : " + s);

        s = DateFormat.getDateInstance(DateFormat.FULL, locale).format(date);
        out.println("DateFormat.FULL : " + s);

        s = DateFormat.getDateInstance(DateFormat.DEFAULT, locale).format(date);
        out.println("DateFormat.DEFAULT : " + s);

        // Parse
        try {
            date = DateFormat.getDateInstance(DateFormat.DEFAULT, locale).parse("Feb 8, 2005");
            out.println("Parsed date : " + date);
        } catch (ParseException e) {
            out.println(e);
        }
    }
}
```

The output will be:

```
DateFormat.SHORT : 2/8/05
DateFormat.MEDIUM : Feb 8, 2005
DateFormat.LONG : February 8, 2005
DateFormat.FULL : Tuesday, February 8, 2005
DateFormat.DEFAULT : Feb 8, 2005
Parsed date : Tue Feb 08 00:00:00 EET 2005
```

Formatting and parsing a number using a default format

`NumberFormat` is the `abstract` base class for all number formats. This class provides the interface for formatting and parsing numbers. `NumberFormat` also provides methods for determining which locales have number formats, and what their names are.

`NumberFormat` helps you to format and parse numbers for any locale. Your code can be completely independent of the locale conventions for decimal points, thousands-separators, or even the particular decimal digits used, or whether the number format is even decimal.

To format a number for the current `Locale`, use one of the factory class methods:

```
myString = NumberFormat.getInstance().format(myNumber);
```

To format a number for a different `Locale`, specify it in the call to `getInstance()`:

```
NumberFormat nf = NumberFormat.getInstance(Locale.US);
```

Use `getInstance()` or `getNumberInstance()` to get the normal number format. Use `getIntegerInstance()` to get an integer number format. Use `getCurrencyInstance` to get the currency number format. And use `getPercentInstance` to get a format for displaying percentages. With this format, a fraction like 0.53 is displayed as 53%.

```
import static java.lang.System.out;
import java.text.DateFormat;
import java.text.NumberFormat;
import java.text.ParseException;

public class NumberFormatExample1 {
```

```
    public static void main(String[] args) {
        // Format
        long iii = 1000;
        Long jjj = 1000L;
        NumberFormat format = NumberFormat.getInstance();
        out.println("Format long : " + format.format(iii));
        out.println("Format Long : " + format.format(jjj));

        // Parse
        try {
            Number nnn1 = format.parse("1000");
            out.println("Parsed Number 1  : " + nnn1);

            Number nnn2 = format.parse("1 000");
            out.println("Parsed Number 2  : " + nnn2);
        } catch (ParseException e) {
            out.println(e);
        }
    }
}
```

The output is:

```
Format long : 1 000
Format Long : 1 000
Parsed Number 1  : 1000
Parsed Number 2  : 1
```

Formatting and parsing a number for a **Locale**

A formatted number consists of locale-specific symbols such as the decimal point. This example demonstrates how to format a number for a particular locale:

```
import static java.lang.System.out;
import java.text.NumberFormat;
import java.text.ParseException;
import java.util.Locale;

public class NumberFormatExample2 {

    public static void main(String[] args) {
        // Format
        double iii = 1000.123;
        Double jjj = 1000.123;

        NumberFormat formatUS = NumberFormat.getInstance(Locale.US);
        out.println("Format double (US): " + formatUS.format(iii));
        out.println("Format Double (US): " + formatUS.format(jjj));

        NumberFormat formatDE = NumberFormat.getInstance(Locale.GERMANY);
        out.println("Format double (DE): " + formatDE.format(iii));
        out.println("Format Double (DE): " + formatDE.format(jjj));

        // Parse
                try {
            Number nnn1 = formatUS.parse("1234.1234");
            out.println("Parsed Number 1 (US) : " + nnn1);

            Number nnn2 = formatUS.parse("1234,1234");
            out.println("Parsed Number 2 (US) : " + nnn2);
```

```
            Number nnn3 = formatDE.parse("1234.1234");
            out.println("Parsed Number 3 (DE) : " + nnn3);

            Number nnn4 = formatDE.parse("1234,1234");
            out.println("Parsed Number 4 (DE) : " + nnn4);
        } catch (ParseException e) {
            out.println(e);
        }
    }
}
```

The output will be:

```
Format double (US): 1,000.123
Format Double (US): 1,000.123
Format double (DE): 1.000,123
Format Double (DE): 1.000,123
Parsed Number 1 (US) : 1234.1234
Parsed Number 2 (US) : 12341234
Parsed Number 3 (DE) : 12341234
Parsed Number 4 (DE) : 1234.1234
```

Formatting and parsing currency

The `NumberFormat` provides 2 factory methods for currency format instances:

```
public static final NumberFormat getCurrencyInstance()
public static NumberFormat getCurrencyInstance(Locale inLocale)
```

```
import static java.lang.System.out;
import java.text.NumberFormat;
import java.text.ParseException;
import java.util.Locale;

public class CurrencyFormatExample {

    public static void main(String[] args) {
        // Format
        double iii = 999.99;

        NumberFormat format = NumberFormat.getCurrencyInstance();
        NumberFormat formatUS = NumberFormat.getCurrencyInstance(Locale.US);
        NumberFormat formatDE = NumberFormat.getCurrencyInstance(Locale.GERMANY);

        out.println("Format currency (default): " + format.format(iii));
        out.println("Format currency (US): " + formatUS.format(iii));
        out.println("Format currency (DE): " + formatDE.format(iii));

        // Parse
        try {
            Number nnn1 = format.parse("1234,12 руб.");
            out.println("Parsed currency 1 : " + nnn1);

            Number nnn2 = formatUS.parse("$1234.12");
            out.println("Parsed currency 2 (US) : " + nnn2);

            Number nnn3 = formatDE.parse("1234,12 €");
            out.println("Parsed currency 3 (DE) : " + nnn3);
```

```
        } catch (ParseException e) {
            out.println(e);
        }
    }
}
```

The output will be similar to the following:

```
Format currency (default): 999,99 руб.
Format currency (US): $999.99
Format currency (DE): 999,99 €
Parsed currency 1 : 1234.12
Parsed currency 2 (US) : 1234.12
Parsed currency 3 (DE) : 1234.12
```

`java.util.Locale` class

A `Locale` object represents a specific geographical, political, or cultural region. An operation that requires a `Locale` to perform its task is called locale-sensitive and uses the `Locale` to tailor information for the user. For example, displaying a number is a locale-sensitive operation - the number should be formatted according to the customs/conventions of the user's native country, region, or culture.

Create a `Locale` object using the constructors in this class:

```
Locale(String language)
Locale(String language, String country)
Locale(String language, String country, String variant)
```

The `Locale` class provides a number of convenient constants that you can use to create `Locale` objects for commonly used locales. For example, the following creates a `Locale` object for the United States:

```
Locale locale = Locale.US;
```

You can get the default locale using `static Locale.getDefault()` method:

```
Locale locale = Locale.getDefault();
```

The Java 2 platform provides a number of classes that perform locale-sensitive operations. For example, the `NumberFormat` class formats numbers, currency, or percentages in a locale-sensitive manner. Classes such as `NumberFormat` have a number of convenience methods for creating a default object of that type. For example, the `NumberFormat` class provides these three convenience methods for creating a default `NumberFormat` object:

```
// default locale
NumberFormat.getInstance()
NumberFormat.getCurrencyInstance()
NumberFormat.getPercentInstance()
```

These methods have two variants; one with an explicit locale and one without; the latter using the default locale.

```
// custom locale
NumberFormat.getInstance(myLocale)
NumberFormat.getCurrencyInstance(myLocale)
NumberFormat.getPercentInstance(myLocale)
```

A `Locale` is the mechanism for identifying the kind of object (`NumberFormat`) that you would like to get. The locale is just a mechanism for identifying objects, not a container for the objects themselves.

Prev

Develop code that serializes and/or de-serializes objects using the following APIs from java.io: DataInputStream, DataOutputStream, FileInputStream, FileOutputStream, ObjectInputStream, ObjectOutputStream and Serializable.

Up

Home

Next

Write code that uses standard J2SE APIs in the java.util and java.util.regex packages to format or parse strings or streams. For strings, write code that uses the Pattern and Matcher classes and the String.split(...) method. Recognize and use regular expression patterns for matching (limited to: . (dot), * (star), + (plus), ?, \d, \s, \w, [], ()). The use of *, +, and ? will be limited to greedy quantifiers, and the parenthesis operator will only be used as a grouping mechanism, not for capturing content during matching. For streams, write code using the Formatter and Scanner classes and the PrintWriter.format/printf methods. Recognize and use formatting parameters (limited to: %b, %c, %d, %f, %s) in format strings.